

Working with Dataframes

Accessing a Dataframe – Base R



```
df[ row , column]
```



```
df[ ,2]
```



```
df$col_name
```

Access a specific column (using name in quotes, position or \$ and column name (no quotes))



```
df[2, ]
```

Access a specific row



```
df[2,2]
```

Access a specific value (by specifying row and column)

Accessing basic dataframe information:

- `nrow(df)` Number of rows
- `ncol(df)` Number of columns
- `dim(df)` Number of rows and columns



pipes

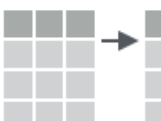
`x %>% f(y)`
becomes `f(x, y)`

For more info, see [dplyr cheatsheet](#)

Functions that target columns



```
select(.data, ...)
```



```
Extract columns as a table.
```



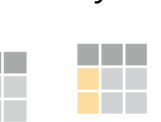
```
select(iris, Sepal.Length, Species)
```



```
mutate(.data, ...)
```



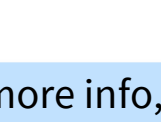
```
Compute new column(s).
```



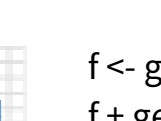
```
mutate(mtcars, gpm = 1/mpg)
```



```
rename(.data, ...)
```



```
Rename columns.
```



```
rename(iris, Length = Sepal.Length)
```

Creating grouped data

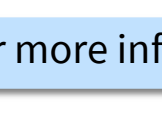
Use `group_by()` to create a "grouped" copy of a table. `dplyr` functions will manipulate each "group" separately and then combine the results.



```
mtcars %>%
```



```
group_by(cyl) %>%
```



```
summarise(avg = mean(mpg))
```

Functions that target rows



```
filter(.data, ...)
```



```
Extract rows that meet logical criteria.
```



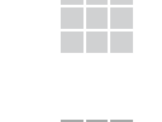
```
filter(iris, Sepal.Length > 7)
```



```
distinct(.data, ..., .keep_all = FALSE)
```



```
Remove rows with duplicate values.
```



```
distinct(iris, Species)
```



```
arrange(.data, ...)
```



```
Order rows by values of a column or columns (low to high), use with desc() to order from high to low.
```



```
arrange(mtcars, mpg)
```

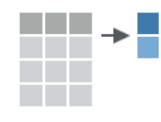


```
arrange(mtcars, desc(mpg))
```

Summarizing data



```
summarise(.data, ...)
```



```
Compute table of summaries.
```



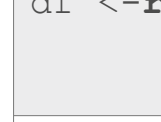
```
summarise(mtcars, avg = mean(mpg))
```



```
count(x, ..., wt = NULL, sort = FALSE)
```

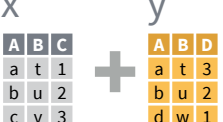


```
Count number of rows in each group defined by the variables in ... Also tally().
```

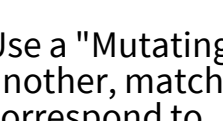


```
count(iris, Species)
```

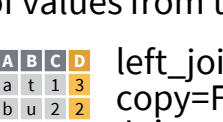
Combining two data sets



```
left_join(x, y, by = NULL,
```



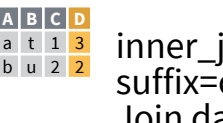
```
copy=FALSE, suffix=c("..x", "..y"),...)
```



```
Join matching values from y to x.
```



```
right_join(x, y, by = NULL, copy = FALSE,
```



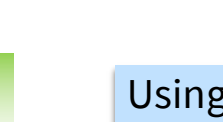
```
suffix=c("..x", "..y"),...)
```



```
Join matching values from x to y.
```



```
inner_join(x, y, by = NULL, copy = FALSE,
```



```
suffix=c("..x", "..y"),...)
```



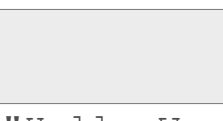
```
Join data. Retain only rows with matches.
```



```
full_join(x, y, by = NULL, copy=FALSE,
```

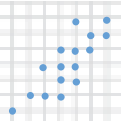


```
suffix=c("..x", "..y"),...)
```

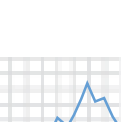


```
Join data. Retain all values, all rows.
```

Some common graphs



```
e <- ggplot(mpg, aes(cty, hwy))
```



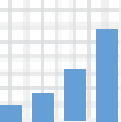
```
e + geom_point()
```



```
x, y, alpha, color, fill, shape, size, stroke
```



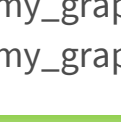
```
i <- ggplot(economics, aes(date, unemploy))
```



```
i + geom_line()
```



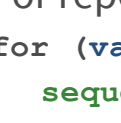
```
x, y, alpha, color, group, linetype, size
```



```
c <- ggplot(mpg, aes(hwy))
```



```
c + geom_histogram(binwidth = 5)
```



```
x, y, alpha, color, fill, linetype, size, weight
```



```
d <- ggplot(mpg, aes(fl))
```



```
d + geom_bar(position = "dodge")
```



```
x, alpha, color, fill, linetype, size, weight
```

Change Labels and title on graph

`my_graph + xlab("x axis label")` # change x axis

`my_graph + ylab("y axis label")` # change y axis

`my_graph + ggtitle("This is my title")` # add title

Other programming concepts

For loops

For repeated tasks

```
for (variable in sequence) { Do something }
```

Example:

```
for (i in 1:4) {  
  j <- i + 10  
  print(j)  
}
```

Custom functions

Automate steps to simplify

```
func_name <-  
  function(var) {  
    Do something  
    return(new_variable)  
  }
```

```
square <- function(x)  
{ squared <- x*x  
  return(squared)  
}
```

Conditional functions

If this, do that..

```
ifelse(test,  
  output if true,  
  output if false)
```

```
ifelse(age >=  
  18, "adult",  
  "child")
```

Multi-case ifelse:

```
case_when(test1 ~ output if true,  
  test2 ~ output if true,  
  test3 ~ output if true)
```

```
case_when(age <= 12 ~ "child",  
  age >= 12 ~ "teenager",  
  age >= 18 ~adult")
```